

1 Tổng quát hoá và chẩn đoán (Generalization and Diagnosis)

PHỤ THUỘC C/12/03-toi-uu-hoa (nhiều gradient, kế toán bộ nhớ của trạng thái Adam); B/04-nen-tang-hoc-tu-du-lieu và B/07-chia-du-lieu-va-danh-gia (vì sao “tổng quát hoá” chỉ có nghĩa khi tập chia đúng).

TÓM TẮT

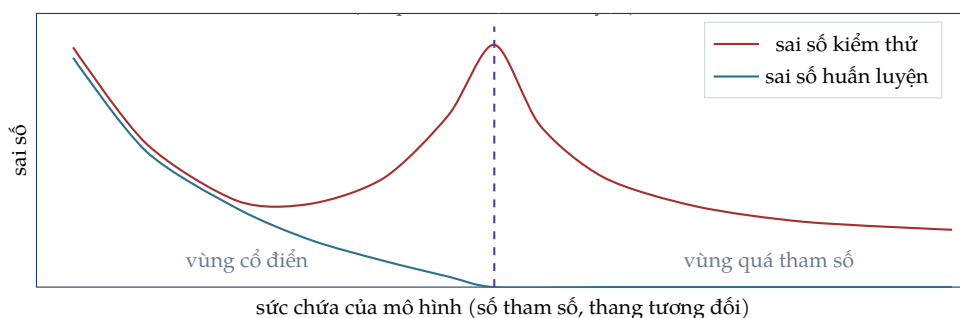
Mục này xử lý khoảng cách giữa “mạng khớp được dữ liệu huấn luyện” và “mạng dùng được”, rồi biến phần còn lại thành quy trình chẩn đoán. Đọc xong bạn xếp được các công cụ **chính quy hoá** (*regularization*) theo hiệu quả thực tế trong thị giác, thay vì theo độ nổi tiếng. Bạn biết đưa mô hình lên nhiều GPU theo đúng thứ tự tăng dần độ phức tạp. Và bạn có một quy trình bắt lỗi cho kết luận trong vài phút thay vì vài ngày. Nếu chỉ nhớ một điều: trước khi chạy full, hãy *overfit một batch 20–50 mẫu tới loss gần 0*. Phép thử này tách bạch “vòng lặp huấn luyện có bug” khỏi “mô hình không tổng quát hoá được”. Phần lớn thời gian bị mất là do người ta nhầm hai thứ đó với nhau.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tập huấn luyện, xác thực, kiểm thử** — ba phần rời nhau của dữ liệu: phần để học, phần để chọn siêu tham số và dùng đúng lúc, phần chỉ dùng đúng một lần để báo cáo. Trộn lẫn ba phần là cách phổ biến nhất để tự lừa mình.
- **overfitting** — mô hình khớp rất tốt dữ liệu huấn luyện nhưng kém trên dữ liệu mới, vì nó đã học cả những chi tiết chỉ đúng với tập huấn luyện.
- **chính quy hoá (regularization)** — mọi kỹ thuật nhằm thu hẹp khoảng cách giữa kết quả huấn luyện và kết quả thực tế — bằng cách ràng buộc mô hình, thêm nhiễu, hoặc thêm thông tin.
- **augmentation** — sinh thêm mẫu huấn luyện bằng cách biến đổi mẫu có sẵn (lật, cắt, đổi màu); mỗi phép biến đổi là một tuyên bố “nhân không đổi dưới phép này”.
- **sức chứa (capacity)** — độ phong phú của lớp hàm mà mô hình biểu diễn được, thường xấp xỉ bằng số tham số; lý thuyết cổ điển gắn sức chứa với overfitting, còn thực nghiệm hiện đại thì không.
- **rò rỉ dữ liệu (data leakage)** — thông tin của tập đánh giá lọt vào quá trình huấn luyện, khiến điểm số đẹp giả tạo — ví dụ hai khung hình liên tiếp của cùng một video nằm ở hai bên ranh giới chia tập.
- **tiền trình và GPU** — trong huấn luyện phân tán, mỗi GPU thường chạy một tiến trình riêng giữ một bản mô hình; “đồng bộ” nghĩa là các tiến trình trao đổi gradient để giữ các bản đó giống nhau.
- **EMA (trung bình trượt lũy thừa)** — một bản trọng số phụ được cập nhật bằng trung bình có trọng số giảm dần theo thời gian; nó mượt hơn trọng số hiện thời và thường cho kết quả đánh giá tốt hơn.

A. Đặt vấn đề

Loss huấn luyện xuống đẹp, độ chính xác trên tập kiểm thử thì không. Đây là bài toán cũ nhất của học máy, nhưng cách hiểu nó trong deep learning đã thay đổi hẳn và việc mang nguyên trực giác cũ sang là một nguồn suy luận sai lớn. Trực giác cổ điển nói: mô hình quá nhiều tham số so với dữ liệu thì học thuộc lòng, nên hãy giảm sức chứa mô hình. Nhưng một mạng hiện đại khớp hoàn hảo được cả một tập ImageNet với nhãn đã bị xáo ngẫu nhiên [1]. Sức chứa của nó dư sức học thuộc. Vậy mà chính nó vẫn tổng quát hoá tốt khi nhân là chân trị. Các cận sai số dựa trên sức chứa do đó gần như rỗng, và hiện tượng *double descent* còn cho thấy tăng sức chứa vượt qua điểm nội suy có thể làm sai số kiểm thử *giảm* trở lại [2], [3].



Hình 1. Hiện tượng giảm kép (*double descent*), vẽ định tính. Trực giác cổ điển chỉ mô tả đúng nửa trái: sai số kiểm thử đi xuống rồi đi lên khi mô hình lớn dần. Nhưng ngay tại ngưỡng nội suy — chỗ mô hình vừa đủ sức khớp hết dữ liệu huấn luyện — sai số kiểm thử đạt *đỉnh* rồi *giảm trở lại* khi ta tiếp tục tăng sức chứa. Đây là lý do câu “mô hình quá lớn thì sẽ overfit” không dùng được làm quy tắc, và là lý do phải chọn công cụ chính quy hoá theo đo đặc chứ không theo lý thuyết sức chứa. Hình là minh hoạ định tính; vị trí đỉnh và độ sâu của lần giảm thứ hai phụ thuộc tác vụ và lượng nhiễu nhĩn.

Hình 1 nói rõ điều mà một câu trích dẫn không nói được: hai trực giác đối nghịch nhau đều đúng, chỉ là đúng ở hai nửa khác nhau của cùng một trục.

Hệ quả thực hành rất cụ thể: đừng chọn công cụ theo lý thuyết sức chứa, hãy chọn theo hiệu quả đo được trong đúng lĩnh vực của bạn. Và vì phần lớn các lần “mô hình không tổng quát hoá” thực ra là “vòng lặp huấn luyện có lỗi”, phần thứ hai của mục này quan trọng không kém phần thứ nhất. Giá trị cụ thể: sau mục này bạn có một cây quyết định để chẩn đoán một lần chạy hỏng, và một bảng triệu chứng để tra thay vì đoán.

↪ **tiền đề** B/07 — khoảng cách train-val chỉ có nghĩa nếu tập được chia đúng; chia sai thì mọi kết luận ở đây sụp.

TRỰC GIÁC

Chính quy hoá thường được kể như “ràng buộc mô hình cho nó bớt phức tạp”. Nhưng công cụ mạnh nhất trong thị giác lại là augmentation, và nó không ràng buộc gì cả: nó *thêm thông tin*. Hãy nghĩ theo kiểu người hiệu chuẩn cảm biến: khi bạn lắc camera qua nhiều tư thế lúc hiệu chuẩn, bạn không làm mô hình camera đơn giản đi, bạn đang nói cho bộ ước lượng biết đại lượng nào *không được phép* thay đổi kết quả. Augmentation làm đúng việc đó với mạng: mỗi phép biến đổi là một câu tuyên bố “nhân không đổi dưới phép này”. Vì thế nó thắng các công cụ ràng buộc sức chứa — nó trả lời một câu hỏi mà chúng không trả lời được.

B. Bộ công cụ chính quy hoá, xếp theo hiệu quả thực tế trong thị giác

Thứ tự dưới đây là thứ tự *hiệu quả thực hành trong computer vision*, không phải thứ tự nổi tiếng. Nó khác đáng kể thứ tự mà sách giáo khoa trình bày.

- **Augmentation** — gần như luôn cho lợi ích lớn nhất trên mỗi giờ công bỏ ra, vì nó là công cụ duy nhất trong danh sách *thêm thông tin* thay vì lấy bớt tự do. Chi tiết ở mục C.
- **Early stopping** — dừng khi loss xác thực bắt đầu xấu. Gần như miễn phí, không có siêu tham số ngoài kiên nhẫn, và trong nhiều recipe nó là thứ đang âm thầm gánh phần lớn công việc chính quy hoá.
- **Weight decay** — vẫn hữu ích, nhưng cơ chế thật của nó trong mạng có normalization không phải cơ chế trong sách. Với một tầng conv đứng ngay trước BatchNorm, nhân W với một hằng số *không đổi* hàm số mà mạng biểu diễn, vì BN chuẩn hoá lại ngay sau đó. Vậy weight decay không thu hẹp lớp hàm; cái nó thật sự điều khiển là *độ lớn của bước học hiệu dụng*, vì

bước tương đối $\|\Delta W\|/\|W\|$ tăng khi $\|W\|$ bị co lại. Đây là lý do weight decay và learning rate không độc lập với nhau như tên gọi gợi ý.

- **EMA của trọng số** — giữ một bản trung bình trượt của trọng số và dùng nó để đánh giá. Rẻ, gần như luôn cải thiện một chút, và cơ chế thì đã nói ở mục trước: SGD ở cuối quá trình lang thang trong một quả cầu bán kính tỉ lệ với η , nên lấy trung bình là đứng vào tâm quả cầu đó thay vì một điểm ngẫu nhiên trên nó.
- **Stochastic depth** — bỏ ngẫu nhiên cả một khối residual trong lúc huấn luyện [4]. Hợp với mạng rất sâu; nó là “dropout ở cấp khối” và không phá thống kê không gian như dropout cấp phần tử.
- **Dropout** — *đã mất phần lớn vai trò* trong CNN thị giác hiện đại [5]. Có hai lý do. BatchNorm đã bơm sẵn nhiễu qua thống kê batch, nên nhiễu thêm là thừa. Và hai kỹ thuật xung khắc nhau: dropout làm phương sai kích hoạt lúc huấn luyện khác lúc suy luận, đúng đại lượng mà BN đang cố cố định. Nó vẫn sống khoẻ ở tầng đầu ra của transformer và ở các mô hình ngôn ngữ.

Điểm đáng nhớ của cả danh sách không phải thứ tự mà là *vi sao* thứ tự đó lệch với sách vở: sách xếp theo mức độ ràng buộc lớp hàm, thực tế xếp theo lượng thông tin mà công cụ mang thêm vào hoặc lượng nhiễu mà nó khớp với chế độ suy luận.

C. Augmentation là nơi bạn phát biểu invariance

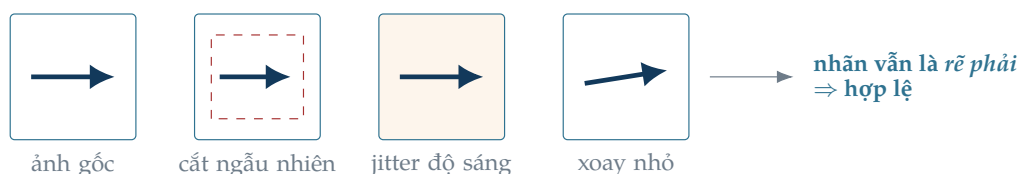
Câu hỏi “augmentation nào là đúng” không có câu trả lời chung, nhưng có một cách nhìn thống nhất làm nó trở thành câu hỏi trả lời được: *mỗi phép augmentation là một tuyên bố về tính bất biến mà bạn muốn mô hình có*. Lật ngang là tuyên bố “lớp không đổi khi lật trái phải”. Jitter màu là tuyên bố về bất biến với điều kiện chiếu sáng. Cắt ngẫu nhiên là tuyên bố “đối tượng có thể ở bất kỳ đâu và bất kỳ tỉ lệ nào”. Từ đó suy ra ngay quy tắc chọn: hãy đưa vào đúng những phép biến đổi thật sự xảy ra trong dữ liệu triển khai của bạn, và loại bỏ những phép phá huỷ nhãn.

Chính vì vậy augmentation có ý nghĩa vật lý mạnh hơn augmentation ngẫu nhiên. Nếu camera của bạn có nhiễu đọc, nhiễu bắn photon và một đường ống ISP cụ thể, thì mô phỏng đúng chuỗi đó (xem A/02-vat-ly-tao-anh) tạo ra các mẫu nằm trên đúng đa tạp mà mô hình sẽ gặp; còn cộng nhiễu Gauss vào ảnh đã qua ISP thì tạo ra các mẫu không tồn tại trong thực tế.

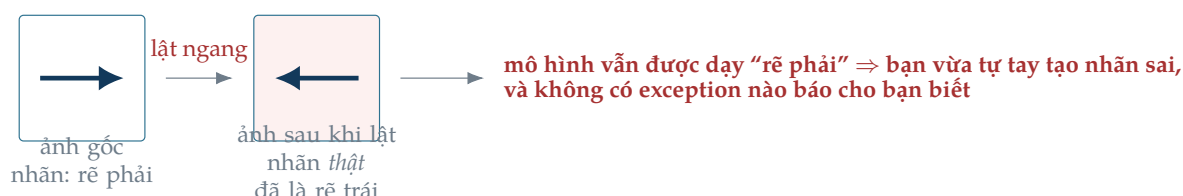
↪ **hệ quả của A/02** — chuỗi tạo ảnh vật lý quyết định phép biến đổi nào sinh ra mẫu có thật, phép nào sinh ra mẫu không bao giờ gặp.

Nhóm hiện đại thì mở rộng ý tưởng theo hai hướng khác nhau: **Mixup** và **CutMix** trộn hai mẫu và trộn cả nhãn theo cùng tỉ lệ [6], [7], tức tuyên bố một giả định về *tính tuyến tính giữa các lớp* chứ không còn về bất biến hình học; **RandAugment** thì tự động hoá việc chọn cường độ và số phép [8], giảm không gian tune xuống hai con số; còn copy-paste và mosaic ghép các đối tượng vào bối cảnh mới, đặc biệt hiệu quả cho detection vì chúng tăng số đối tượng mỗi ảnh.

Giữ nhãn — mỗi phép này là một tuyên bố “lớp không đổi dưới phép biến đổi này”



Phá nhãn — cùng một phép, sai bài toán



Hình 2. Augmentation là nơi bạn phát biểu tính bất biến, nên nó chỉ đúng khi tuyên bố đó đúng. Hàng trên: bốn phép biến đổi không đụng tới thứ quyết định nhãn, nên chúng dạy mô hình bỏ qua các chiều biến thiên vô nghĩa. Hàng dưới: cùng một phép lật ngang, nhưng ở bài toán mà *hướng* chính là lớp, nó tạo ra một cặp ảnh–nhãn mâu thuẫn. Đây cũng là lý do việc chọn augmentation phải bắt đầu từ câu hỏi “cái gì được phép đổi mà nhãn không đổi”, chứ không phải từ danh sách mặc định của thư viện.

Hình 2 cho thấy vì sao không tồn tại một bộ augmentation “tốt” chung: cùng một phép biến đổi vừa là công cụ mạnh nhất vừa là nguồn nhãn sai, tùy vào bài toán.

Chỗ hỏng thì rất cụ thể và đáng thuộc: augmentation phá nhãn. Lật ngang một ảnh chữ số hoặc biến báo rẽ trái là tạo ra một mẫu có nhãn sai. Jitter màu trong bài toán mà màu *chính là* lớp (đèn giao thông, phân loại quả chín) cũng vậy. Xoay lớn trong ảnh trên cao thì hợp lý, trong ảnh chụp bởi camera gắn xe thì không, vì trọng lực là một tiên nghiệm thật.

D. Từ một GPU tới nhiều GPU

Bốn bước dưới đây xếp theo đúng thứ tự tăng dần độ phức tạp và mức tiết kiệm bộ nhớ. Nguyên tắc là *chỉ leo lên bước sau khi bước trước đã hết dư địa*. Trước khi đọc bảng, hãy cố định bảng kế toán. Với P tham số huấn luyện ở fp32 bằng Adam, mỗi GPU phải giữ $4P$ byte trọng số, $4P$ byte gradient và $8P$ byte trạng thái Adam. Tổng là $16P$ byte, tức 4,8 GB khi $P = 300$ triệu — trước khi tính một byte kích hoạt nào.

Bảng 1. Bốn mức song song hoá dữ liệu. Cột bộ nhớ tính cho N tiến trình và bỏ qua kích hoạt; cột cuối là cột đáng đọc kỹ nhất.

Bước	Cơ chế	Bộ nhớ mỗi GPU	Hỏng khi nào
DataParallel	Một tiến trình, phát tán mô hình sang các GPU mỗi bước	$16P$	Nghẽn ở GIL và ở việc sao chép mô hình; GPU 0 quá tải. Coi như đã lỗi thời
DDP	Mỗi GPU một tiến trình, all-reduce gradient sau backward	$16P$	Seed và sampler không đặt đúng thì các tiến trình thấy cùng dữ liệu; BatchNorm không đồng bộ nên mỗi GPU là một mô hình hơi khác
Gradient accumulation	Chia batch thành micro-batch, cộng dồn gradient rồi mới bước	$16P$, nhưng kích hoạt giảm theo micro-batch	Không giảm bộ nhớ trạng thái; thống kê BatchNorm vẫn tính theo micro-batch nên batch hiệu dụng lớn mà thống kê thì không
ZeRO / FSDP [9]	Chia nhỏ trạng thái (giai đoạn 1), gradient (2), rồi cả trọng số (3) giữa các tiến trình	$8P + 8P/N$ tới $16P/N$	Lưu lượng giao tiếp tăng theo giai đoạn; nghẽn khi mạng liên kết chậm hoặc mô hình nhiều tầng nhỏ

Ba cạm bẫy trong Bảng 1 đáng nêu tên riêng vì chúng không gây lỗi mà chỉ gây kết quả kém. Thứ nhất là *seed và sampler*: nếu mọi tiến trình dùng cùng seed cho việc xáo dữ liệu, bạn không huấn luyện trên N lần dữ liệu mà trên cùng một dòng dữ liệu N lần. Thứ hai là *BatchNorm*. Mỗi tiến trình tính thống kê trên batch cục bộ của nó. Với detection chạy một tới hai ảnh mỗi GPU, thống kê nhiều tới mức vô dụng. Đây chính là lý do GroupNorm và SyncBN tồn tại, và cũng là vấn đề đã gặp ở mục 2 của chương này. Thứ ba là *cách gộp loss*: lấy tổng thay vì trung bình làm learning rate hiệu dụng bị nhân với số tiến trình, một lỗi im lặng có hậu quả đúng bằng việc đặt sai learning rate một bậc.

↪ **hệ quả của** C/12/03 — bảng kế toán $16P$ byte cho trọng số, gradient và trạng thái Adam là lý do tồn tại của ZeRO

Song song với bốn bước trên còn một trục thứ hai đã nhắc ở mục 1: **gradient checkpointing** [10] đối khoảng 30 phần trăm thời gian tính lấy việc giảm bộ nhớ kích hoạt từ $O(L)$ xuống $O(\sqrt{L})$. Trong thị giác, kích hoạt thường lớn hơn trọng số rất nhiều, nên với một GPU đơn thì checkpointing hầu như luôn là bước nên thử trước ZeRO.

E. Chính quy hoá ngầm: chỗ ngành chưa có câu trả lời

Cần biết mục này để *không suy luận sai*, chứ không phải để dùng. Câu hỏi mở là: vì sao một mạng quá tham số, đủ sức học thuộc nhiều, lại chọn đúng nghiệm tổng quát hoá được khi ta huấn luyện nó bằng SGD? Các mảnh câu trả lời hiện có đều là mảnh: nhiều của SGD thiên vị các nghiệm ở vùng phẳng của mặt mất mát; các thuật toán bậc nhất có xu hướng tìm nghiệm chuẩn nhỏ trong các bài toán tuyến tính quá tham số; khởi tạo và lịch học đều để lại dấu vết trên nghiệm cuối. Chưa có lý thuyết nào ghép các mảnh đó lại thành một lời giải thích đầy đủ cho mạng sâu thật.

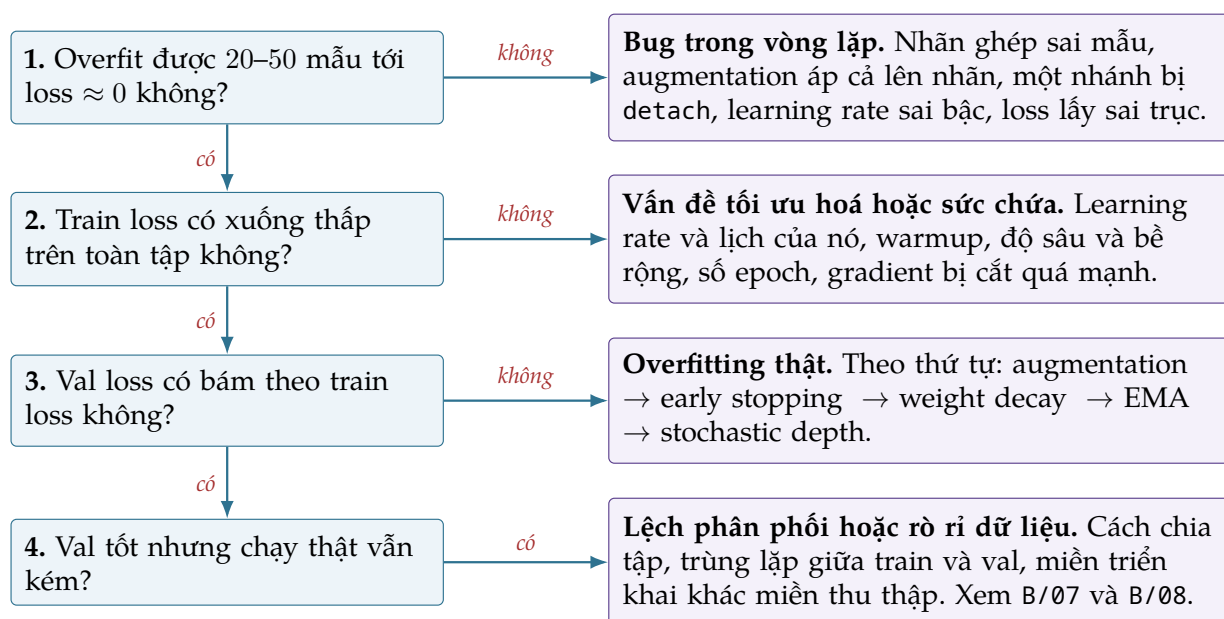
Hệ quả cho cách bạn đọc và nói: đừng kết luận “mô hình này quá lớn nên sẽ overfit” như một mệnh đề chắc chắn, vì nó thường sai trên thực nghiệm; và cũng đừng kết luận ngược lại rằng sức chứa không quan trọng. Phát biểu trung thực là: quá trình huấn luyện, chứ không chỉ lớp hàm, quyết định nghiệm — và ta chưa mô tả được “chứ không chỉ” đó một cách định lượng.

F. Phản xạ debug số một: overfit một batch nhỏ

Trước khi chạy full, lấy 20 tới 50 mẫu, tắt mọi augmentation và mọi chính quy hoá, rồi huấn luyện tới khi loss huấn luyện gần 0. Nếu không đạt được điều đó, dừng lại: bug nằm trong vòng lặp, không nằm ở khả năng tổng quát hoá, và mọi phút bỏ ra để tune siêu tham số lúc này là phí.

Sức mạnh của phép thử nằm ở chỗ nó *loại bỏ đúng các biến gây nhiễu*. Trên 20 mẫu, một mạng có hàng triệu tham số thừa sức nhớ hết, nên nếu nó không nhớ được thì tín hiệu không đi từ dữ liệu tới loss hoặc gradient không đi ngược lại được. Nó kiểm tra toàn tuyến — đọc dữ liệu, tiền xử lý, ghép nhãn, hàm mất mát, backward, bước cập nhật — trong vài phút, thay vì để bạn suy đoán từ một đường cong sau ba giờ. Đây cũng là lý do nó phải là việc đầu tiên, không phải việc cuối cùng.

Hình 3 biến phép thử đó thành một cây quyết định đầy đủ. Điểm đáng chú ý là mỗi nhánh dẫn tới một *họ* nguyên nhân khác hẳn nhau, nên trả lời sai câu hỏi đầu tiên khiến bạn đi tìm ở nhầm họ suốt phần còn lại.



Hình 3. Cây chẩn đoán một lần chạy không đạt kỳ vọng. Bốn câu hỏi phải hỏi theo đúng thứ tự này, vì mỗi câu chỉ có nghĩa khi câu trước đã trả lời “có”; bỏ qua câu số một là nguyên nhân phổ biến nhất của việc tune siêu tham số cho một hệ thống đang có bug.

G. Phá hỏng có chủ đích: xây thư viện triệu chứng

Kỹ năng cao nhất của cả chương không phải biết cách sửa mà là biết *nhìn một đường cong và đoán ra nguyên nhân*. Kỹ năng đó không đến từ đọc; nó đến từ việc cố ý làm hỏng từng thứ một và ghi lại hình dạng đường cong tương ứng. Bảng 2 là tám phiên bản hỏng đáng chạy, và cột cuối là cột biến bảng này từ một danh sách thành một công cụ chẩn đoán.

Bảng 2. Tám lỗi cố ý và triệu chứng của chúng. Chạy cả tám, vẽ tám đường cong vào một hình duy nhất, và giữ hình đó.

Phá hỏng	Triệu chứng trên đường cong	Phân biệt bằng phép đo nào
Learning rate $\times 100$	Loss vọt lên rồi thành NaN trong vài chục bước, hoặc phẳng ngay ở mức phỏng đoán ngẫu nhiên	Tỉ lệ update trên trọng số ở bước đầu vượt 10^{-1} ; chuẩn gradient tăng theo bước
Learning rate $\div 100$	Loss giảm gần như tuyến tính, không có dấu hiệu bão hoà khi hết lịch	Tỉ lệ update trên trọng số dưới 10^{-5}
Quên shuffle	Loss răng cưa với chu kỳ đúng bằng một epoch	Vẽ loss theo từng bước thay vì theo epoch; hình răng cưa lộ ra ngay
Quên eval()	Val loss thấp bất thường và đôi khi đổi batch size lúc đánh giá	Chạy val hai lần với batch 1 và batch 64, so kết quả
Nhân lệch một chỉ số	Train loss giảm bình thường; val accuracy quanh mức ngẫu nhiên	Ma trận nhầm lẫn lệch hẳn khỏi đường chéo một cột
Quên chuẩn hoá đầu vào	Hội tụ chậm hơn rõ rệt; nếu có BatchNorm thì gần như không thấy gì	Histogram của batch đầu vào; thống kê kích hoạt tầng đầu
Khởi tạo toàn 0	Loss đứng đúng mức ngẫu nhiên, không nhích	Các hàng của W giống hệt nhau sau vài bước: đối xứng chưa bị phá
Dropout 0,9	Train loss cao hơn val loss; cả hai hội tụ rất chậm	Khoảng cách train trừ val mang dấu âm — điều gần như không bao giờ xảy ra khi mọi thứ đúng

H. Giới hạn và trường hợp hỏng

Phép thử “overfit một batch nhỏ” có một chế độ hỏng đáng biết, và nó là loại chế độ hỏng khó chịu nhất: phép thử *qua* trong khi hệ thống vẫn sai. Nếu nhân của bạn bị hoán vị theo một quy tắc cố định — ví dụ chỉ số lệch một — thì trên 20 mẫu, mạng học thuộc ánh xạ sai đó dễ dàng và loss vẫn về 0. Cùng lý do, một bug trong tiền xử lý áp giống nhau cho mọi mẫu cũng không bị bắt. Nói cách khác, phép thử chứng minh “tín hiệu chảy được từ đầu vào tới loss và ngược lại”, không chứng minh “tín hiệu chảy đúng thứ”. Bổ sung rẻ nhất là nhìn trực tiếp vài chục mẫu đã qua toàn bộ pipeline, kèm nhãn, ngay trước khi vào mô hình.

↪ **tiền đề** E/24 — ablation chạy trên một hệ thống còn bug là vô nghĩa, nên phép thử này phải đứng trước mọi thí nghiệm so sánh

Bảng triệu chứng cũng có giới hạn: nó được lập trên một cấu hình cụ thể, và một số triệu chứng bị normalization che mất — “quên chuẩn hoá đầu vào” gần như vô hình khi mạng có BatchNorm ở tầng đầu. Đây là mặt trái đã nói ở mục 2: normalization làm hệ thống bền hơn với lỗi và vì thế xoá bớt các triệu chứng mà bạn muốn học đọc. Cuối cùng, thứ tự ưu tiên các công cụ chính quy hoá ở mục B chỉ là *kinh nghiệm thực hành trong thị giác*. Nó được tổng hợp từ các recipe phổ biến, không phải từ một nghiên cứu so sánh có kiểm soát. Ở lĩnh vực khác, chẳng hạn mô hình ngôn ngữ, thứ tự đó khác.

CẠM BẮY

Rõ ràng dữ liệu là cách phổ biến nhất để có một đường cong xác thực đẹp và một hệ thống vô dụng, và nó gần như không bao giờ trông giống một lỗi. Ba dạng hay gặp trong thị giác: chia tập theo khung hình thay vì theo chuỗi, khiến hai khung hình cách nhau 1/30 giây nằm ở hai bên ranh giới; augmentation hoặc chuẩn hoá được tính trên *toàn bộ* dữ liệu trước khi chia; và các ảnh trùng lặp hoặc gần trùng nằm ở cả hai phía. Dấu hiệu nhận biết: khoảng cách train–val nhỏ bất thường, và điểm số tụt mạnh ngay khi bạn đánh giá trên dữ liệu thu ở một buổi khác.

CẦN NHỚ

Trước khi tune bất cứ thứ gì, overfit 20–50 mẫu tới loss gần 0. Đây là ranh giới tách “vòng lặp có bug” khỏi “mô hình không tổng quát hoá được”. Hai họ nguyên nhân đó cần hai bộ công cụ hoàn toàn khác nhau. Nhắm chúng với nhau là cách tốn thời gian nhất trong nghề. [T1]

I. Kết nối

Mục này khép lại chương 12 và mở sang hai hướng. Về phía dữ liệu, “tổng quát hoá” chỉ có nghĩa khi tập được chia đúng, nên B/07-chia-du-lieu-va-danh-gia là tiền đề bắt buộc của cả mục, và B/08-dich-chuyen-phan-phoi là chương giải thích nhánh cuối cùng của cây chẩn đoán. Về phía kiến trúc, thư viện triệu chứng ở mục G là công cụ bạn sẽ dùng liên tục khi đọc C/13-kien-truc-backbone. Bài học lớn nhất của chương đó là ConvNeXt: phần lớn chênh lệch giữa các kiến trúc hoá ra là chênh lệch công thức huấn luyện. Bài học ấy chỉ có nghĩa nếu bạn phân biệt được hai nguồn. Cuối cùng, sự nghi ngờ có kỷ luật với một bảng so sánh là nội dung của E/24-thi-nghiem-va-ablation.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao lập luận “mô hình nhiều tham số hơn số mẫu thì sẽ học thuộc lòng” không còn dùng được, và điều đó thay đổi cách bạn chọn công cụ chính quy hoá thể nào?
2. Trong một mạng có BatchNorm, weight decay thật sự điều khiển đại lượng nào? Vì sao nó không thu hẹp lớp hàm?
3. Augmentation nào bạn phải *bỏ* nếu bài toán là phân loại đèn giao thông, và vì sao?
4. Gradient accumulation cho batch hiệu dụng lớn hơn nhưng không sửa được một vấn đề của BatchNorm. Vấn đề đó là gì?
5. Phép thử “overfit một batch nhỏ” qua được nhưng hệ thống vẫn sai — tình huống cụ thể nào gây ra chuyện đó, và phép kiểm tra bổ sung nào bắt được nó?
6. Bạn thấy train loss cao hơn val loss — hai nguyên nhân khả dĩ là gì?
7. ZeRO giai đoạn 3 giảm bộ nhớ xuống $16P/N$; nó đổi lấy cái gì, và khi nào phép đổi đó không đáng?

Nguồn

- [1] C. Zhang, S. Bengio, M. Hardt, et al., “Understanding deep learning requires rethinking generalization,” in *ICLR*, 2017.
- [2] M. Belkin, D. Hsu, S. Ma, et al., “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS*, vol. 116, no. 32, pp. 15 849–15 854, 2019.
- [3] P. Nakkiran, G. Kaplun, Y. Bansal, et al., “Deep Double Descent: Where Bigger Models and More Data Hurt,” in *ICLR*, 2020.
- [4] G. Huang, Y. Sun, Z. Liu, et al., “Deep Networks with Stochastic Depth,” in *ECCV*, 2016.
- [5] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [6] H. Zhang, M. Cisse, Y. N. Dauphin, et al., “mixup: Beyond Empirical Risk Minimization,” in *ICLR*, 2018.
- [7] S. Yun, D. Han, S. J. Oh, et al., “CutMix: Regularization Strategy to Train Strong Classifiers,” in *ICCV*, 2019.
- [8] E. D. Cubuk, B. Zoph, J. Shlens, et al., “RandAugment: Practical Automated Data Augmentation,” in *CVPR Workshops*, 2020.
- [9] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [10] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.